

Architectural and UX Reconstruction of Google+ (2015-2019): A Comprehensive Guide for Open-Source Preservation

1. The 2015-2019 era, precisely

To accurately reconstruct Google+ as it existed between 2015 and 2019, it is necessary to first decouple the platform's architecture and user experience from its 2011 origins. The defining characteristic of the late-stage Google+ era was a strategic, systemic pivot away from mapping real-world interpersonal relationships (a "social graph" paradigm dominated by Facebook) toward mapping topical affinities and shared passions (an "interest graph" paradigm)¹.

This ideological shift culminated in a massive visual and architectural overhaul launched on November 18, 2015, spearheaded by Product Director Luke Wroblewski and the broader design team⁴. The redesign brutally simplified the platform, jettisoning heavy legacy features to focus the user experience almost entirely on two core paradigms: Communities (group-based topical discussions) and Collections (single-user curated content feeds)⁶.

The user interface underwent a radical transformation heavily influenced by the nascent Material Design 1 (MD1) specification. The underlying codebase was rewritten to prioritize mobile responsiveness and rendering speed, slashing the initial desktop home page weight from a bloated 22,600 KB down to a highly performant 327 KB⁵. The redesign featured a responsive, card-based interface that dynamically scaled from small mobile screens to large desktop monitors, presenting content in one or two columns depending on viewport width⁶.

Crucially, the hallmark feature of early Google+—"Circles"—was aggressively de-emphasized. While Circles still existed in the backend data model for granular privacy controls and targeted audience sharing, the iconic drag-and-drop UI was replaced by a simplified checkbox interface. Furthermore, the dedicated Circle stream was hidden from the primary navigation by default, requiring users to navigate into advanced settings to explicitly re-enable it⁴. Other prominent features, such as Google Photos and Hangouts, were decoupled entirely from the social network, becoming standalone services⁵. The stream itself began utilizing machine learning algorithms to surface content based on user engagement with specific topics, moving away from a strictly chronological feed toward a personalized interest graph¹.

The target sub-era for a faithful, stable reconstruction should be anchored precisely in **mid-2017**.

The reasoning for this specific architectural anchor point is twofold. First, the initial November 2015 rollout was technically a preview and lacked several core features that power users relied upon. Throughout 2016, the engineering team iteratively reintroduced missing features, such as Events, and deployed improved community moderation tools, including the ability to hold posts for review and sort community feeds by relevance⁴. By mid-2017, the feature set had reached

its maximum stability and completeness for this era.

Second, the platform entered a terminal decline in early 2018 following an internal audit named "Project Strobe." This audit uncovered a critical vulnerability in the Google+ People API (present since 2015) that allowed third-party developers to access the non-public profile fields of up to 500,000 users¹³. Fearing regulatory scrutiny akin to the concurrent Facebook Cambridge Analytica scandal, Alphabet Inc. patched the bug internally in March 2018 without public disclosure, eventually announcing the consumer shutdown of the network in October 2018¹³. From early 2018 until the final shutdown in April 2019, the platform was effectively in maintenance mode, heavily overrun by spam, and abandoned by its primary developers¹⁸. Therefore, targeting the mid-2017 window captures the redesign at its most functional, polished, and actively maintained state.

2. Source material

Reconstructing a defunct, closed-source distributed system requires forensic analysis of scattered digital artifacts. While the live platform no longer exists, the structural, relational, and visual DNA of the 2015-2019 era is preserved across several distinct archival vectors.

The most comprehensive raw repository of the platform's final state is the 1.6-petabyte dataset collected by the Archive Team (currently hosted on Archive.org) shortly before the April 2019 shutdown²⁰. This dataset captured millions of public posts, Communities, and user profiles.

While primarily consisting of raw HTML and WARC (Web ARChive) files, this repository provides an invaluable reference for the exact Document Object Model (DOM) structure, CSS class naming conventions, and responsive layout logic of the web client during the target era. Analyzing these WARC files will allow a developer to extract the exact SVG paths used for iconography and the precise hex codes used for the interface.

For backend data modeling, Google Takeout exports from former users serve as the definitive schema reference. When Google+ was deprecated, users were permitted to download their historical data²¹. This data was exported in well-structured JSON format, explicitly delineating how the platform modeled relationships²¹. The ActivityLog directory within these archives contained precise JSON schemas for arrays of +1s on posts, +1s on comments, Comments, Poll Votes, and Posts²⁴. By analyzing these JSON structures, a database architect can reverse-engineer the exact relational data models utilized by Google, understanding precisely how a Comment object references a Post object, and how Collections were structurally distinguished from Communities at the database level.

Because the 2015 redesign was heavily driven by Luke Wroblewski, his public presentations and blog posts from 2015 and 2016 act as primary source design documents¹¹. Furthermore, the exact user interface components used in the redesign are perfectly preserved in the original Google Material Design 1 specifications. These specifications provide pixel-perfect measurements for card padding (e.g., 16dp spacing), elevation shadows, and typography baselines²⁷.

Despite these resources, reconstruction will encounter severe friction due to sparse documentation in two specific areas. First, the exact machine-learning weights and algorithmic

logic used to curate the "interest graph" home feed¹ remain proprietary and undocumented. A reconstruction will need to substitute this with a chronological or simplified edge-rank algorithm. Second, while the static Google Takeout JSON provides the data schema at rest, the exact structure of the dynamic REST/RPC payloads sent between the 2017 client and server is largely lost, save for glimpses in third-party API documentation or archived StackOverflow questions³². The reconstruction will require the invention of custom API contracts to facilitate frontend-backend communication.

3. Naming and branding, given this context

The legal and cultural precedents for fan-made recreations of discontinued software platforms present a clear dichotomy: projects that attempt to usurp intellectual property face aggressive legal termination, while projects that evoke the spirit of a platform through distinct, independent branding thrive.

The most instructive cautionary tale is *Club Penguin Rewritten*, a fan-made clone of Disney's defunct virtual world. Despite operating strictly as a non-commercial passion project, its direct use of Disney's copyrighted assets, characters, and trademarks led to a total shutdown orchestrated by corporate lawyers, resulting in domain seizures and law enforcement involvement³⁵. Conversely, *SpaceHey* serves as the model for success. Created as a meticulous homage to the 2005-era MySpace, SpaceHey successfully captured millions of users by replicating the HTML/CSS structure, the profile customization features, and the general aesthetic of MySpace. Critically, it used a distinct name, original logos, and made no claim to MySpace intellectual property³⁸.

Given the context of a public GitHub repository and a live portfolio instance intended for peer review, utilizing the name "Google+", the "G+" logo, or Google's proprietary branding carries an unacceptable risk of a Cease and Desist (C&D) order from Alphabet Inc. Trademark law requires corporations to actively defend their marks to retain them; a public clone practically invites automated legal scrutiny.

The branding strategy must balance discoverability for nostalgia-seekers against absolute legal independence. The project name should abstractly reference the platform's core paradigms (Circles, Collections, Sparks, or Streams) without referencing the parent company.

Proposed Branding Concept	Rationale and Connection to the 2015-2019 Era
Project Sparks	A direct nod to an early, beloved Google+ feature that curated content based on interests, which was later spiritually succeeded by Collections ¹⁹ .
The Interest Graph	Reflects the fundamental architectural and ideological pivot of the 2015 redesign ¹ .

PlusInfinity	Abstracts the "+" symbol, retaining the phonetic association without utilizing trademarked nomenclature.
Collections & Circles	Directly names the two most defining organizational structures of the platform's history.

The visual identity should heavily leverage the open-source Material Design language. By using a deep crimson or vermillion primary color combined with the standard Material Design floating action buttons (FAB) and typography (Roboto), the interface will immediately trigger the intended psychological nostalgia for former users, all while remaining entirely legally distinct from Google's trademarked assets.

4. Pixel-faithful vs. spirit-faithful

A critical architectural decision for a historical reconstruction is whether to pursue a "pixel-faithful" replication of the 2017 Document Object Model (DOM) or a "spirit-faithful" adaptation that updates the interface to modern standards while retaining the original interaction paradigms.

The 2015 redesign, while universally praised for its rendering speed and mobile-centric approach, was highly controversial among power users utilizing desktop environments. The transition from a dense, three-column layout to a sparse, two-column layout introduced excessive "white space creep" on wider monitors⁷. Furthermore, the interaction pattern for reading comments forced users to click a post to open it in a full-window modal, breaking the ability to seamlessly scroll and consume content⁷.

From a technical perspective, a strict pixel-faithful recreation on modern hardware presents significant user experience friction. The 2015 interface was designed for device resolutions and aspect ratios that are now antiquated.

The trade-offs between the two approaches are distinct. Recreating the exact 2017 user interface guarantees absolute nostalgia but requires intentionally implementing outdated UX anti-patterns. It requires utilizing the exact Material Design 1 elevation specifications, where a resting card sits at exactly 2dp of elevation and a Floating Action Button (FAB) sits at 6dp²⁷. Conversely, a spirit-faithful approach layers modern UI polish (e.g., Material Design 3's tonal elevations, dark modes, and fluid layouts) over the original information architecture⁴¹. It resolves the desktop whitespace issues by utilizing responsive CSS Grid layouts that take full advantage of modern ultra-wide monitors.

The most effective strategy for a portfolio piece is a hybrid approach: spirit-faithful layout mechanics combined with pixel-faithful component styling.

To trigger the precise nostalgic "feel" of Google+, the micro-interactions must strictly adhere to the Material Design 1 specification. Cards must possess a hard drop shadow rather than the soft tonal shifts of modern UI paradigms⁴³. The FAB must predictably transition from 6dp to

12dp upon hover or press²⁷. However, macro-layouts—how the cards flow on a 2026 mobile screen versus a 4K monitor—should be spirit-faithful, utilizing modern CSS clamp() functions and CSS Grid auto-fit properties to ensure the application feels like a professional, highly capable piece of software rather than a broken, static relic.

UI Element	2015-2019 MD1 Specification (Pixel-Faithful)	Modern Recommendation (Spirit-Faithful Adaptation)
Content Cards	2dp resting shadow, zero border radius, hard borders ²⁷ .	Retain the 2dp shadow for nostalgia, but update internal padding to modern accessibility standards.
Stream Layout	Fixed 1 or 2 columns, resulting in excessive whitespace on 1080p+ displays ⁷ .	Utilize CSS Grid auto-fit to dynamically scale columns based on the viewport, optimizing screen real estate.
Comment Threads	Full-window modal takeover, preventing simultaneous feed scrolling ⁷ .	Implement inline expansion or a sliding side-panel to maintain context within the main feed.
Floating Action Button	6dp resting elevation, 12dp pressed elevation, hard color fills ²⁷ .	Retain exact MD1 elevation metrics and transitions, as this is critical to the tactile feel of the era.

5. The empty-room problem

A perfectly engineered social network is functionally useless without a populated network graph. When a user logs into a freshly deployed, isolated platform clone, an empty home stream instantly shatters the illusion of revival. To solve this "empty-room problem," the platform requires immediate data seeding mechanisms and strategic community outreach to generate initial velocity.

The most powerful tool for initial data seeding is the user's own historical data. The application should feature a prominent "Restore from Google Takeout" utility. By parsing the structured JSON files (Posts.json, Comments.json, ActivityLog.json) from a user's uploaded Takeout archive²¹, the system can instantly repopulate their profile, recreate their Collections, and

backfill their historical stream. This transforms the application from a mere structural clone into a functional, interactive archive of the user's past, providing immediate value even in a single-player context.

However, to solve the empty stream problem beyond a single user's isolated archive, the system architecture must embrace protocol-level federation. Following the Google+ shutdown, large contingents of its user base migrated to decentralized platforms like Mastodon, Hubzilla, and Diaspora, while others moved to closed alternatives like MeWe⁴⁵.

By implementing the ActivityPub Server-to-Server (S2S) protocol natively in the backend, the clone can instantly tap into the existing "Fediverse"⁴⁹. ActivityPub is a decentralized social networking protocol based on the ActivityStreams 2.0 data format⁴⁹. It allows the isolated Google+ instance to subscribe to content streams from millions of existing users across Mastodon, Lemmy, and other compatible platforms⁵². A user on the Google+ clone could create a "Collection," and the backend would generate an ActivityPub Announce or Create JSON-LD payload⁵¹, broadcasting it to followers residing on remote Mastodon servers. Conversely, inbound federated posts can be parsed, styled, and rendered as MD1 Google+ cards in the user's stream. This architectural decision guarantees that the network is alive with fresh, interactive content from day one.

To attract the specific demographic of former power users to the live instance, outreach should be highly targeted to the digital enclaves where they currently reside. The subreddit r/googleplus, though small, serves as a concentrated gathering point for nostalgic users. Furthermore, the platform MeWe heavily advertised itself as the privacy-focused alternative to Google+ during the 2018-2019 shutdown window, successfully onboarding millions of refugees⁴⁵. Identifying tech-focused groups on MeWe and sharing the open-source repository will likely yield enthusiastic early adopters. Finally, the Fediverse itself represents a highly receptive audience; instances dedicated to open-source software and technology history (e.g., fosstodon.org) are primed for a federated Google+ revival.

6. Feature scope for solo build

The 2015-2019 iteration of Google+ was a massive, globally distributed system maintained by hundreds of engineers and researchers. A solo developer must ruthlessly prioritize features based on a matrix comparing their essentiality to the nostalgic experience against their architectural complexity.

The overarching design philosophy of the 2015 redesign was the primacy of the "interest graph" over the "social graph"². The platform pivoted to become a destination for exploring passions rather than managing real-world friendships. Therefore, features that facilitate topical curation must take absolute precedence over features that facilitate direct interpersonal communication.

Google+ Feature	Nostalgia Value (2015-2017 Era)	Engineering Complexity	Recommendation for Version 1.0
-----------------	------------------------------------	------------------------	-----------------------------------

Collections	Essential. The core of the 2015 pivot, acting as single-user curated topic feeds ⁸ .	Low. Requires a simple one-to-many relational mapping of posts to a specific Collection ID.	Build. This is the foundational element of the interest graph.
Communities	Essential. The primary interaction hub for group-based discussions and shared passions ⁶ .	Medium. Requires role-based access control (RBAC) for moderators, join requests, and distinct content feeds.	Build. Critical for multi-user engagement.
Streams (Home Feed)	Essential. The primary consumption interface for the user.	High. Requires complex timeline generation, infinite scroll pagination, and aggregate querying.	Build (Simplified). Rely on chronological sorting rather than complex machine learning algorithms.
Profile / About	High. The identity anchor for the user within the ecosystem.	Low. Standard CRUD operations and basic image hosting.	Build.
Notifications	High. The notification bell was iconic and drives user retention.	Medium. Requires WebSocket or Server-Sent Events (SSE) implementation for real-time delivery.	Build (Simplified). Push events for mentions and likes only.
Circles	Medium. Actively de-emphasized in the 2015 UI and reduced to a backend filtering mechanism ⁴ .	Extreme. Requires highly complex intersection queries for Access Control Lists (ACLs) and dynamic feed	Cut for v1.0. Implement standard unidirectional followers first, adding list-based filtering later.

		filtering.	
Hangouts / Video	Low. Spun off into a separate, standalone application prior to the redesign ⁵ .	Extreme. Requires WebRTC implementation, STUN/TURN server provisioning, and heavy bandwidth scaling.	Cut completely. Out of scope for both the era and a solo portfolio project.

The most significant architectural concession for a solo build must be the algorithmic feed. In the late-stage era, Google+ utilized sophisticated graph neural networks and machine learning to analyze user engagement, parse the bipartite user-item interest graph, and dynamically sort the stream to surface relevant content¹. Attempting to replicate this level of multi-scale attention and contrastive learning³¹ is entirely beyond the scope of a non-commercial solo project. The feed generation logic must revert to a strictly chronological reverse-time query, filtered solely by explicit follows (users, Collections, or Communities).

7. Tech stack

The unique constraints of this project—solo development, open-source portfolio display, zero-to-low budget, and the need for rapid iteration against a fixed, well-documented specification—demand a stack that abstracts infrastructure management while providing robust relational data modeling capabilities. While Java/Spring Boot is highly capable, a full-stack JavaScript/TypeScript approach minimizes context switching, leverages generous modern serverless tiers, and aligns perfectly with the requirement for free or near-free hosting.

Frontend and Backend Framework: Next.js (App Router) & TypeScript Next.js provides server-side rendering (SSR) capabilities that are crucial for the Search Engine Optimization (SEO) of public Communities and Collections, mimicking the fast time-to-first-byte that was heavily advertised in the 2015 redesign⁶. TypeScript enforces the rigid data schemas required to handle complex nested objects, which is critical when parsing unpredictable JSON-LD payloads from federated ActivityPub servers. Next.js Server Actions handle the standard CRUD operations for posts and profiles natively within the ecosystem, while standard API routes provide the necessary endpoints to handle inbound WebHooks and the ActivityPub S2S protocol⁵⁷.

Styling: Tailwind CSS Tailwind allows for the rapid creation of a custom design system that perfectly mirrors the Material Design 1 specifications. The developer can define custom utility classes in the configuration file to replicate the exact MD1 drop shadows (e.g., shadow-md1-card, shadow-md1-fab-pressed), ensuring pixel-perfect adherence to the historical aesthetic without writing bloated custom CSS²⁷.

Database and Authentication: Supabase (PostgreSQL) Supabase serves as the ideal engine for this project. It provides a robust, open-source PostgreSQL database on a generous free tier.

Crucially, it handles complex authentication (including social logins and JSON Web Tokens) out of the box. PostgreSQL's relational nature is perfectly suited for modeling the heavily interconnected graph of Users, Posts, Communities, and Collections.

Realtime Updates: Supabase Realtime To simulate the instant, reactive feel of the Google+ notification bell, Supabase Realtime allows the Next.js frontend to subscribe directly to PostgreSQL database changes (e.g., tracking a new row in the notifications table) via WebSockets, ensuring UI optimistic updates are seamlessly synchronized with backend state⁵⁹.

Media Storage: Cloudflare R2 Image hosting is traditionally the most expensive and complex aspect of a social network. Cloudflare R2 offers 10 GB of free storage per month and, critically, zero egress fees⁶⁰. The Next.js server can generate a short-lived presigned URL, allowing the client browser to upload image assets directly to the R2 bucket, completely bypassing the Next.js server and saving massive amounts of compute bandwidth⁶¹.

Hosting: Vercel or Cloudflare Pages For a non-commercial portfolio project, hosting the Next.js application on Vercel's free tier or Cloudflare Pages provides automated CI/CD pipelines directly from the public GitHub repository. These platforms offer edge caching and serverless function execution at zero cost for low-traffic applications.

Video Integration (If Required): LiveKit While Hangouts was spun off into a separate product and is recommended to be cut from the scope, if video integration is strictly desired to showcase technical breadth, building raw WebRTC is ill-advised. Implementing LiveKit (an open-source WebRTC infrastructure platform) via their free Cloud tier allows for the integration of multi-party video rooms into the Next.js frontend with minimal infrastructure overhead, successfully simulating the classic Hangouts experience.

8. Realistic scale and light safety baseline

Even at a hobbyist scale, launching a live platform where users can post publicly and federate via ActivityPub exposes the instance to bot networks, spam, and malicious actors. The 2015-2018 era of Google+ was famously overrun by spam due to inadequate automated moderation, a factor that actively degraded the user experience and accelerated the platform's demise¹⁹.

Because Supabase exposes the PostgreSQL database directly to the frontend via its client libraries, data security must be strictly enforced at the database level using Row Level Security (RLS)⁶⁴. RLS ensures that a user can only SELECT, UPDATE, or DELETE rows that they have cryptographic authorization to access.

However, in a social network architecture, naive RLS policies can easily cause catastrophic performance bottlenecks⁶⁷. For example, an RLS policy that checks a blocks table (e.g., a rule stating "Do not render this post if the author is blocked by the requesting user") requires the database engine to execute a subquery for every single post evaluated in the stream⁶⁸. In a

stream of 100 posts, this results in $O(N^2)$ execution latency, potentially causing the connection pool to stall entirely.

To mitigate this performance degradation, the architecture must utilize Security Definer Functions in PostgreSQL⁶⁴. Instead of evaluating complex joins on every single row during a

query, a Security Definer function can fetch the user's block list into a session variable once per transaction. This allows the RLS policy to execute against a cached variable in $O(1)$ time, maintaining strict security without sacrificing the query speed required for a snappy feed⁷². For a solo instance, building automated content moderation tools is unfeasible. Instead, the safety baseline must rely on friction and user-driven curation:

1. **Invite-Only Registration:** To prevent automated bot signups, the platform should launch as an invite-only system. Registration should require a valid cryptographic token generated by the administrator.
2. **Federation Filtering:** ActivityPub introduces the risk of inbound spam from rogue instances. The server must implement domain-level blocking (defederation) to silence noisy or explicitly malicious servers at the network edge⁵³.
3. **Muting and Blocking:** Users must have the ability to explicitly block other users. At the database level, a blocked user's ID is appended to an array, and the query planner drops all edges connecting the two nodes in the graph, ensuring mutual invisibility.

9. Honest risk check

While building a historical reconstruction is a technically enriching portfolio project, several specific architectural and psychological risks threaten to stall development or result in an unusable product.

The most prominent technical risk lies in the ActivityPub protocol itself. While ActivityPub elegantly solves the empty-room problem by plugging the instance into the Fediverse, the protocol is notoriously vague. As noted by protocol implementers, "much of its behaviour is undefined"⁵³. The core types lack strict behavioral constraints, leading to a fragmented ecosystem where different platforms (Mastodon, Lemmy, Pleroma) interpret payloads differently. For example, the specification lacks a formalized mechanism for direct messages, leading to competing, incompatible standards that require custom parsing logic⁵³. A solo developer risks losing months in "implementation hell," endlessly debugging raw JSON-LD payloads just to get federated comments to render correctly. The mitigation strategy is to strictly implement a read-only or highly constrained subset of ActivityPub for version 1.0, rather than attempting full bidirectional compatibility.

The second major technical hurdle is database performance related to graph traversal. Social networks are inherently graph structures. Mapping a graph relationship (Users -> Follows -> Collections -> Posts -> Comments) onto a relational database like PostgreSQL requires complex multi-way JOIN operations or recursive Common Table Expressions (CTEs). As the database grows, querying a chronological home feed generated from hundreds of followed entities will degrade logarithmically. The developer must be highly proficient in indexing foreign keys and analyzing EXPLAIN ANALYZE outputs, or the platform will suffer from severe latency spikes that ruin the user experience⁷².

Finally, the project faces a significant psychological risk: the "rose-tinted glasses" effect. Nostalgia often obscures usability flaws. The 2015-2019 era of Google+ was not universally beloved; it was heavily criticized for confusing navigation patterns, hidden settings (like the

buried Circle stream), and poor use of desktop real estate⁵. By committing to a completely faithful recreation of the era, the developer risks inheriting the exact user experience friction that alienated users in 2016. It is vital to recognize when to break from historical accuracy in favor of modern UX conventions, ensuring the project remains a functional homage rather than a flawed museum piece.

Works cited

1. Meet The New Google+: Simpler, Faster, Better, <https://deepakravlani.blogspot.com/2015/11/meet-new-google-simpler-faster-better.html>
2. Characterization of information propagation in Google+ and its, <http://mirage.cs.uoregon.edu/pub/Ripples-2.pdf>
3. Tracing the Birth of an OSN: Social Graph and Profile Analysis in, <https://schmiste.github.io/websci12.pdf>
4. Google+ - Wikipedia, <https://en.wikipedia.org/wiki/Google%2B>
5. Your Guide to the Brand New Redesign of Google+, <https://www.elegantthemes.com/blog/resources/your-guide-to-the-brand-new-redesign-of-google>
6. Google+ looks a little different today... - iDownloadBlog, <https://www.idownloadblog.com/2015/11/18/google-plus-redesign/>
7. Looking at the revamped Google+ | ***Dave Does the Blog, <https://hill-kleerup.org/blog/2015/11/18/looking-at-the-revamped-google.html>
8. Google+ has been completely redesigned with a focus ... - Engadget, <https://www.engadget.com/2015-11-17-google-plus-redesign-communities-collections.html>
9. Google+ Changes: What Marketers Need to Know, <https://www.socialmediaexaminer.com/google-plus-changes-what-marketers-need-to-know/>
10. Why you should stop thinking Google+ is dead - Blu Mint, <https://blumint.co/blog/stop-thinking-google-plus-dead>
11. The Year in Review 2015: Google for Content Creators, Collectors, <https://www.peggyketc.com/2015/12/2015-year-in-review.html>
12. Year in Review 2016: Google+ - Creators, Collections and, <https://www.peggyketc.com/2016/12/year-in-review-2016-google-creators.html>
13. Goodbye Google+ - Tech Success, <https://techsuccess.com.au/goodbye-google-plus/>
14. Google+ is Shutting Down After a Vulnerability ... - The Hacker News, <https://thehackernews.com/2018/10/google-plus-shutdown.html>
15. Project Strobe: Protecting your data, improving our third-party APIs, <https://blog.google/innovation-and-ai/technology/safety-security/project-strobe/>
16. Google Plus is being shuttered – why? - Silicon Republic, <https://www.siliconrepublic.com/enterprise/google-bug-shutdown>
17. Google shutting down Google+ after exposing data of up to ... - CNET, <https://www.cnet.com/tech/tech-industry/google-reportedly-exposed-data-of-hundr>

- [eds-of-thousands-of-google-users/](#)
18. Google+ Social Network: Rise and Fall Explained | iNet Ventures,
<https://inetventures.com/blog/flashback-what-happened-to-the-google-social-network>
 19. The Fall of Google+ - The Vital Edge by Gideon Rosenblatt,
<https://www.the-vital-edge.com/fall-of-google-plus/>
 20. The Web as History - Pure,
https://pure.au.dk/ws/files/111004966/The_Web_as_History.pdf
 21. Google Takeout: Export a copy of your data in Google services,
<https://news.ycombinator.com/item?id=25066838>
 22. FiXato/Plexodus-Tools - GitHub, <https://github.com/FiXato/Plexodus-Tools>
 23. How to export and backup your Google Hangouts chat history,
<https://blog.jay2k1.com/2014/11/10/how-to-export-and-backup-your-google-hangouts-chat-history/comment-page-6/>
 24. Comments - Voidstar,
<https://www.voidstar.com/Takeout/ActivityLog/Comments.html>
 25. Google+ Redesigned | Hacker News,
<https://news.ycombinator.com/item?id=12391430>
 26. LukeW | Writings on Digital Product Strategy & Design, <https://www.lukew.com/ff/>
 27. Elevation & shadows - Material Design,
<https://m1.material.io/material-design/elevation-shadows.html>
 28. Material Design: Introduction, <https://m1.material.io/>
 29. Material Design Layout Metrics Guide | PDF - Scribd,
<https://www.scribd.com/document/859934427/Metrics-keylines-Layout-Material-Design>
 30. DiffNet++: A Neural Influence and Interest Diffusion Network for,
<https://www.computer.org/csdl/journal/tk/2022/10/09311623/1q0BylPc0Ew>
 31. (PDF) A graph neural network recommendation algorithm based on,
https://www.researchgate.net/publication/395166274_A_graph_neural_network_recommendation_algorithm_based_on_multi-scale_attention_and_contrastive_learning
 32. Method: activities.watch | Admin console - Google for Developers,
<https://developers.google.com/workspace/admin/reports/reference/rest/v1/activities/watch>
 33. Method: activities.list | Admin console - Google for Developers,
<https://developers.google.com/workspace/admin/reports/reference/rest/v1/activities/list>
 34. Iteration through json with multiple API calls for other requests,
<https://stackoverflow.com/questions/47934314/iteration-through-json-with-multiple-api-calls-for-other-requests>
 35. Interviews – Club Penguin Mountains,
<https://clubpenguinmountains.com/category/mountains-monday/interviews/>
 36. Screwed by the Lawyers - All The Tropes,
https://allthetropes.org/wiki/Screwed_by_the_Lawyers
 37. OvershadowedByControversy / Video Games - TV Tropes,

- <https://tvtropes.org/pmwiki/pmwiki.php/OvershadowedByControversy/VideoGames>
38. Case 143: This service is only available for use individually and non,
<https://edit.tosdr.org/cases/143>
39. Happy to report this is a draf | Video published by Madison Ruff May,
<https://www.lemon8-app.com/@madisonruffmay/7561807729703174686?region=us>
40. Humanizing Science and Engineering,
<https://issues.org/wp-content/uploads/2024/01/Issues-391-Fall-2022.pdf>
41. Elevation – Material Design 3, <https://m3.material.io/styles/elevation>
42. Elevation – Material Design 3,
<https://m3.material.io/styles/elevation/applying-elevation>
43. Elevation - Material Design,
<https://m2.material.io/develop/web/supporting/elevation>
44. Cards - Material Design, <https://m2.material.io/components/cards>
45. social media - Google Docs,
https://docs.google.com/document/d/1lwRVOINiyiXyA0t-u_dkdaPXFfaGieJ1RT9a2ERzQik/edit?usp=sharing
46. Educational Leaders, Professional Learning Communities, and,
<https://digitalcommons.uncfsu.edu/cgi/viewcontent.cgi?article=1208&context=jri>
47. Shutting Down Google+ for Consumers - Hacker News,
<https://news.ycombinator.com/item?id=18169243>
48. Now that Google+ is going to die, why not move the RPG ... - Reddit,
https://www.reddit.com/r/rpg/comments/9mq2n5/now_that_google_is_going_to_die_why_not_move_the/
49. ActivityPub - W3C, <https://www.w3.org/TR/activitypub/>
50. Decentralized Social Networks. Comparing federated and peer-to,
<https://medium.com/decentralized-web/decentralized-social-networks-e5a7a2603f53>
51. arXiv:2412.09011v1 [cs.SI] 12 Dec 2024, <https://arxiv.org/pdf/2412.09011>
52. Social media protocols comparison | Paul Stephen Borile,
<https://www.paulstephenborile.com/2024/11/social-media-protocols-comparison/>
53. ActivityPub: The Good, the Bad and the Ugly,
<https://chrastecky.dev/technology/activity-pub-the-good-the-bad-and-the-ugly>
54. A Graph Neural Network Social Recommendation Algorithm ... - MDPI,
<https://www.mdpi.com/2079-9292/12/6/1477>
55. Simple Social Recommendation with Graph Contrastive Learning,
<https://www.mdpi.com/2227-7390/12/7/1107>
56. How to upload images to CloudFlare R2 in Next.js - YouTube,
<https://m.youtube.com/live/ZPw0jrwfAeA>
57. Activity Pub vs Web Frameworks - Dan Palmer,
<https://danpalmer.me/2023-01-08-activitypub-vs-web-frameworks/>
58. ActivityPub Client API: A Way Forward - Steve Bate,
<https://www.stevebate.net/activitypub-client-api-a-way-forward/>
59. Optimistic UI Patterns with Next.js Server Actions and Supabase,
https://dev.to/mahdi_benrhouma_fe1c6005/optimistic-ui-patterns-with-nextjs-serve

[r-actions-and-supabase-realtime-7e0](#)

60. Cloudflare R2 Pricing: Costs, Savings, and Alternatives in 2026,
<https://filebase.com/blog/cloudflare-r2-pricing-costs-savings-and-alternatives-in-2026/>
61. Presigned URLs · Cloudflare R2 docs,
<https://developers.cloudflare.com/r2/api/s3/presigned-urls/>
62. Direct-to-R2 file uploads in Next.js with presigned URLs,
<https://ch1337.com/blog/presigned-uploads-r2-nextjs>
63. How to Upload Files to Cloudflare R2 in a Next.js App,
<https://www.buildwithmatija.com/blog/how-to-upload-files-to-cloudflare-r2-nextjs>
64. Row Level Security performance | Supabase Docs,
<https://supabase.com/docs/guides/database/postgres/row-level-security-performance>
65. Row Level Security | Supabase Docs,
<https://supabase.com/docs/guides/database/postgres/row-level-security>
66. Supabase Row Level Security (RLS): Complete Guide for 2026,
<https://www.itpathsolutions.com/supabase-row-level-security>
67. RLS Performance and Best Practices - Supabase,
<https://supabase.com/docs/guides/troubleshooting/rls-performance-and-best-practices-Z5Jjwv>
68. Row Level Security Blocker — Solved! | by Ebubeokerezi - Medium,
<https://medium.com/@ebubeokerezi/row-level-security-blocker-solved-a5977fda5de7>
69. Optimize RLS Policies for Performance - Postgres Best Practice,
<https://supaexplorer.com/best-practices/supabase-postgres/security-rls-performance/>
70. Optimizing Postgres Row Level Security (RLS) for Performance,
<https://scottpierce.dev/posts/optimizing-postgres-rls/>
71. Optimizing RLS Performance with Supabase(postgres) - Medium,
<https://medium.com/@antstack/optimizing-rls-performance-with-supabase-postgres-fa4e2b6e196d>
72. Mastering PostgreSQL Row-Level Security (RLS) for Rock-Solid, Multi-Tenancy/
<https://ricofritzsche.me/mastering-postgresql-row-level-security-rls-for-rock-solid-multi-tenancy/>
73. PostgreSQL RLS Query Performance Tuning - CodexiLab,
<https://codexilab.com/blog/postgresql-rls-tuning>